Layered prompt architectures for production LLM agents

The strongest pattern emerging across production agent systems is a hybrid architecture: a small, persistent system prompt (~3K tokens) anchored at the top for caching, combined with on-demand reference material retrieval and structured workspace memory that survives context compaction. This approach, validated by Claude Code, Manus, Cursor, and OpenHands, reduces token costs by 45–90% while maintaining instruction adherence. The critical insight from the research is that every token added to context is a bet against accuracy — production systems aggressively minimize what's loaded while ensuring critical constraints persist through multiple defense layers.

The user's proposed 5-layer architecture aligns well with industry consensus but requires specific ordering optimizations for prompt caching and robust compaction-survival mechanisms for pinned constraints. This report covers each of the six research questions with production evidence, then synthesizes cross-cutting recommendations.

---

1. How production agents actually structure their prompts

Production agent systems universally decompose prompts into layers, though they differ substantially in how many layers and how dynamically they assemble them.

Claude Code represents the most sophisticated layered architecture in production. It dynamically assembles **110+ prompt strings** into a system prompt of ~15,000–20,000 tokens, drawn from distinct categories: a core behavior prompt (~2,972 tokens), 16 tool descriptions with supplemental notes, sub-agent prompts (Explore, Plan, Task agents each get their own system prompts), contextual system reminders, utility agent prompts (conversation summarization, security review, sentiment analysis), and conditional extensions GitHub (learning mode, MCP CLI). GitHub Reference material flows through CLAUDE.md files loaded at session start, Claude while a dedicated conversation compaction agent summarizes history when the context window fills. github The system never compacts the system prompt itself — only message history gets summarized.

Cursor Agent takes a minimalist approach: a single system prompt with **XML-tagged sections** (<communication>, <tool_calling>, <search_and_reading>, <making_code_changes>, <debugging>, <calling_external_apis>) github totaling roughly 57 lines of core rules. Prompt Hacker Reference material is handled through an on-demand fetch_rules tool — rules listed in an <available_instructions> section are loaded only when the model decides they're relevant. github Cursor has **no persistent memory between sessions**. Geeky Gadgets Notably, it employs a two-model architecture where the main model proposes edits and a simpler "apply model" executes them. github

Devin implements the most elaborate autonomy loop with three explicit modes (planning, standard, edit) indicated before each action turn. Its prompt includes an agile-style planning rubric, Git/GitHub workflow instructions, and embedded resource limits (CPU, RAM, runtime quotas). Prompt Hacker Devin's distinguishing feature is **Playbooks** — user-defined instruction sets that persist across sessions for repeated tasks. Substack Cognition Labs found that Claude Sonnet 4.5 "treats the file system as its memory without prompting," frequently writing CHANGELOG.md and SUMMARY.md files for its own future reference.

OpenHands introduces the **microagent pattern** — YAML-configured instruction modules with three trigger types: always-on (repository-wide conventions), keyword-triggered (activated when specific terms appear), and manual (slash-command style). Microagents can optionally provide additional tools via MCP servers. Openhands GitHub This is the closest production parallel to the user's phase-based architecture.

Manus explicitly uses the **file system as structured external memory**, creating a todo.md file that gets continuously rewritten during task execution. A typical Manus task requires ~50 tool calls, and by constantly rewriting the todo list, Manus pushes its objectives into the model's recent attention span — a deliberate attention manipulation mechanism that counters "lost-in-the-middle" effects. manus Manus describes five context engineering dimensions: offloading, reduction, retrieval, isolation, and caching.

SWE-agent provides a counterpoint to complex architectures. Its mini-SWE-agent successor scores >74% on SWE-bench Verified with just ~100 lines of Python and a minimal system prompt. Its philosophy — "focus on the LM, not the scaffold" — suggests that over-engineering prompt layers can be counterproductive for capable models. GitHub

The universal patterns across all systems include XML/tagged sections for structural clarity, tool-first design with JSON schemas, Prompt Hacker two-model architectures separating planning from execution, convention files per repository (CLAUDE.md, .cursorrules, .windsurfrules, .openhands/microagents/), and explicit "never reveal system prompt" clauses. Prompt Hacker

Recommendations for the 5-layer architecture

The user's 5-layer design maps cleanly to production patterns. Layer 1 (system prompt) should follow Claude Code's approach of keeping identity, guardrails, and tool usage policies in a persistent system message that is never compacted. Layer 2 (phase prompts) mirrors Devin's explicit mode system GitHub and OpenHands' microagent triggers. Layer 4 (reference material) should adopt Cursor's on-demand fetch_rules pattern rather than always-loading. The key risk is over-loading: Aider's documentation warns that "above about 25,000 tokens of context, most models start to become distracted."

---

2. On-demand retrieval outperforms upfront loading, but agents forget to retrieve

The research consensus strongly favors **adaptive, on-demand retrieval** over loading everything upfront — but with critical caveats about reliability.

FLARE (Forward-Looking Active Retrieval, EMNLP 2023) generates a tentative next sentence, checks token-level confidence, and triggers retrieval only when low-confidence tokens are detected. arXiv It outperforms both fixed-interval retrieval and no-retrieval baselines across four benchmarks. ACL Anthology **Self-RAG** (2023) trains the model to generate special reflection tokens that adaptively decide when to retrieve, arXiv achieving **81% accuracy on fact-checking** versus 71% for baselines. Hugging Face Both papers demonstrate that indiscriminate retrieval "diminishes LM versatility or can lead to unhelpful response generation." arXiv

The quantified cost savings are dramatic. One production deployment reported reducing tokens from **150K to 2K per session** (98% reduction), dropping costs from $4.50 to $0.06. William Zujkowski Multi-agent RAG research confirms task-specific context retrieval outperforms universal loading by **15–40% on accuracy**.

However, the failure modes are severe. Vercel's agent evaluation found that **skills were never invoked in 56% of test cases**, producing zero improvement over baseline — demonstrating that agents frequently fail to trigger retrieval when they should. The Kamiwaza KAMI benchmark analyzed 900 execution traces and found agents frequently "guess schemas instead of inspecting them," leading to silent failures. Kamiwaza AnythingLLM documentation confirms that models sometimes fabricate tool call outputs rather than actually calling tools. AnythingLLM The TicToc dataset (2025) showed no model achieved better than **65% normalized alignment** with human temporal perception for deciding when cached information is stale. arXiv

The emerging best practice is a **three-tier progressive disclosure pattern**, now adopted by Claude Code, GitHub Copilot, and VS Code as an open standard:

- **Tier 1 (always loaded)**: YAML frontmatter with skill/document name and description only
- **Tier 2 (loaded on activation)**: Full instruction body when the agent or system decides the skill is relevant
- **Tier 3 (loaded on reference)**: Additional files, scripts, and detailed data

Recommendations for the 5-layer architecture

For Layer 4 (reference material), adopt the progressive disclosure pattern. Store reference material in workspace files with descriptive filenames and a manifest listing available resources. Include the manifest (names + one-line descriptions) in the system prompt so the agent knows what exists. Provide a read_reference tool for on-demand loading. For **critical reference material** that must not be missed (e.g., output format templates, methodology requirements), use system-triggered injection — the orchestrator loads them when entering specific phases rather than trusting the agent to retrieve them. This hybrid approach matches Anthropic's recommendation: "retrieve some data up front for speed, let agent explore further autonomously." Anthropic anthropic Never trust the agent alone for mission-critical retrieval — the 56% non-invocation rate is too high.

---

3. Workspace memory patterns converge on structured files with compaction protection

The dominant workspace memory pattern across production systems is **structured markdown files with explicit sections**, combined with compaction strategies that protect critical information.

MemGPT/Letta provides the theoretical foundation, treating the context window as RAM and external storage as disk. arXiv Memgpt Its key innovation is self-editing memory through tool calls: Letta core_memory_replace, core_memory_append, archival_memory_insert, archival_memory_search. The agent itself decides what to evict to external storage and what to summarize, using recursive summarization when the FIFO queue nears capacity. Serokell Information Matters Letta supports custom memory "blocks" (persona, human, task) as core memory persisting across turns. Leonie Monigatti

Claude Code's compaction is the best-documented production implementation. When the context window fills, a dedicated summarization agent compresses message history while preserving "architectural decisions, unresolved bugs, and implementation details" and discarding "redundant tool outputs." github anthropic After compaction, the agent continues with compressed context plus the five most recently accessed files. anthropic Critically, **the system prompt is never compacted** — only conversation history. CLAUDE.md content,

injected as a user message, is loaded fresh at session start (Builder.io) but could be compressed in very long sessions.

Manus's todo.md pattern is perhaps the most clever workspace memory approach. By continuously rewriting the todo list during a ~50-tool-call task, Manus exploits recency bias to keep its global plan in the model's active attention. (manus) Manus also implements restorable compression — tool results can be dropped from context as long as the file path or URL is preserved, allowing re-retrieval. (Medium) (manus) Their five-dimensional context engineering framework (offload, reduce, retrieve, isolate, cache) provides a useful mental model.

Factory.ai's anchored iterative summarization uses explicit sections (session intent, file modifications, decisions made, next steps) that force preservation during compaction. (Factory) On artifact probes like "which files have we modified?", Factory scored **3.6 versus OpenAI's 2.8**, because Factory's summary explicitly lists files in a dedicated section while OpenAI's compression discards file paths as "low-entropy content." (Factory) This validates that **structure forces preservation** — freeform summarization loses critical details.

LangGraph implements four context strategies: Write (scratchpads), Select (RAG and semantic tool selection), Compress (summarization middleware that auto-triggers at 95% context usage), and Isolate (sub-agents with clean context windows). Its (SummarizationMiddleware) and (@dynamic_prompt) decorator modify system prompts based on state before each model call.

Recommendations for the 5-layer architecture

Model workspace.md on Factory.ai's anchored sections pattern. Define explicit sections that the compaction prompt must populate or explicitly leave empty: Pinned Instructions, Current Task State, Decisions Made, Active Constraints, Files Modified, and Next Steps. Implement Manus's restorable compression — when compacting tool results, replace verbose outputs with file paths that can be re-read. Use LangGraph's (SummarizationMiddleware) with a custom compaction prompt that instructs: "Preserve the Pinned Instructions and Active Constraints sections verbatim. Summarize all other sections." Set compaction to trigger at **80% context capacity** (not 95%) to leave room for the agent to continue working after compaction.

---

4. Auto-seeded constraints survive through multi-layer defense

The pattern of auto-seeding critical constraints from configuration is validated by multiple production systems, though no single mechanism guarantees survival — defense in depth is required.

Claude Code's (**--append-system-prompt**) appends content directly to the end of the base system prompt, making it higher priority than CLAUDE.md (which is injected as a user message). Since system prompts are never compacted in Claude Code, this is the strongest position for inviolable constraints. This maps directly to the user's concept of pinned instructions.

ChatGPT Custom Instructions persist 1,500 characters as part of the system prompt for every API call. Saved memories appear under a "# Model Set Context" heading injected after the system prompt in future conversations. This is auto-seeded from user configuration — the same pattern the user is proposing.

However, research on system prompt robustness reveals concerning fragility. A study (arXiv 2502.12197) found that "system message precedence is not directly programmed but learned by models with supervised or

reinforcement learning. Therefore this behavior is susceptible to incidental errors or adversarial manipulation." (arXiv) The "Control Illusion" paper (arXiv 2502.15851) tested six SOTA models and found that the system/user prompt separation **fails to establish a reliable instruction hierarchy** — models exhibit strong inherent biases toward certain constraint types regardless of priority designation. (arXiv) (ResearchGate)

The recommended multi-layer defense for auto-seeded constraints:

- **Layer A — System prompt**: Include a meta-instruction: "Before any response, the PINNED INSTRUCTIONS in workspace.md are inviolable constraints that override all other guidance."
- **Layer B — workspace.md pinned section**: Auto-generate from YAML config at job initialization with clear delimiters ((## ⚠️ PINNED INSTRUCTIONS (DO NOT REMOVE OR SUMMARIZE)))
- **Layer C — Custom compaction prompt**: Explicitly instruct the compaction agent to preserve the pinned section verbatim while summarizing everything else
- **Layer D — Re-injection after compaction**: The harness re-reads the YAML config and re-injects pinned instructions after each compaction event (belt-and-suspenders)
- **Layer E — Output validation**: Post-generation programmatic check against critical constraints (e.g., verify every claim has a citation)

This mirrors how NeMo Guardrails and Guardrails AI operate as runtime enforcement layers independent of context (Medium) — even if a constraint is lost from context, a guardrail wrapper catches violations in output. **Combining prompt-level constraints with output-level validation provides true defense in depth.**

Recommendations for the 5-layer architecture

Auto-seed from YAML config at job initialization — this is validated by ChatGPT, Claude Code, and Windsurf's rules system. Place the most critical constraints in the system prompt itself (Layer 1), not just workspace.md (Layer 3). Structure workspace.md with a clearly delimited pinned section using Factory.ai's anchored approach. (Factory) Implement re-injection from config after every compaction event. Add lightweight programmatic output validation for the most critical constraints (e.g., citation checking). This five-defense approach makes constraint loss require five simultaneous failures.

---

5. Prompt caching demands strict ordering from static to dynamic

Prompt caching provides **45–90% cost reduction and up to 85% latency improvement** when the architecture is correctly ordered, but naive implementations can paradoxically increase costs.

Anthropic's prompt caching requires explicit (cache_control: {"type": "ephemeral"}) markers with up to **4 cache breakpoints** per request. The cache serialization order is fixed: (tools → system → messages). Changes at any level invalidate that level and all subsequent levels. (claude) Minimum cacheable tokens range from 1,024 (Sonnet/Opus 4.x) to 4,096 (Haiku 4.5). (claude) The default **5-minute TTL** refreshes on each hit; an optional 1-hour TTL costs 2× base input price for writes. Cache reads cost just **0.1× base input price** — a 90% discount. Cache writes cost 1.25× base input price, (claude) making the break-even point approximately 2 cache hits per prefix.

OpenAI's prompt caching is fully automatic with no code changes required. Medium The minimum is **1,024 tokens**, matching in 128-token increments. Medium OpenAI Cache reads get a **50% discount** with no write premium, making it immediately beneficial on first hit. OpenAI Developer Community… The typical eviction window is 5–10 minutes of inactivity. OpenAI

vLLM's Automatic Prefix Caching is enabled by default in v1+ and uses hash-based block matching with LRU eviction. Squeezebits However, benchmarks show a **36.7% throughput reduction** on datasets without shared prefixes, Squeezebits so it should only be enabled when requests actually share prefixes (as in agent systems with consistent system prompts).

The "Don't Break the Cache" research paper (arXiv 2601.06007) found that **system-prompt-only caching** provides the most consistent benefits, while **full-context caching can paradoxically increase latency** because naively caching everything triggers cache writes for dynamic content that won't be reused. arXiv

Manus's production experience provides the most actionable guidance: KV-cache hit rate is "the single most important metric for a production-stage AI agent." manus Their input-to-output token ratio is **100:1**, making prefix caching savings transformative. Manus manus Key rules from Manus: never include timestamps at the beginning of the system prompt (kills cache hit rate); make context append-only (never modify previous actions or observations); use deterministic JSON serialization (sort_keys=True); and never dynamically add/remove tools mid-iteration — use token logit masking instead. Galileo AI Manus

Optimal ordering for the 5-layer architecture

The recommended ordering, from most static to most dynamic, with Anthropic cache breakpoints:

1. **Tool definitions** → cache breakpoint #1 (rarely change within a session)
2. **System prompt / Layer 1** (identity, guardrails) → cache breakpoint #2 (persistent across all calls)
3. **Reference material / Layer 4** (templates, methodology) → cache breakpoint #3 (stable within session once loaded)
4. **Phase prompt + Workspace memory / Layers 2+3** → cache breakpoint #4 (stable within a phase)
5. **Task + feedback / Layer 5** → NO breakpoint (most dynamic, always at end)

This ordering ensures that when a phase changes, only layers 4–5 are invalidated. When workspace memory updates, only layer 5 is invalidated. The system prompt and tool definitions achieve near-100% cache hit rates. For the user's vLLM backends running OSS models, enable prefix caching and use session-ID-based routing to ensure requests hit the same GPU — standard load balancers destroy cache locality. manus

---

6. Failure modes demand structured mitigation, not just good prompting

The research reveals several systematic failure modes that threaten layered prompt architectures, each requiring specific architectural countermeasures.

Instruction coherence loss across turns is severe. The landmark "LLMs Get Lost in Multi-Turn Conversation" study (Microsoft Research, May 2025) tested 200,000+ simulated conversations across 15 LLMs

and found a **39% average performance drop** when instructions are split across multiple turns versus given all at once. AI Models arXiv Even flagship models (GPT-4.1, Gemini 2.5 Pro, Claude 3.7 Sonnet) lost 30–40%. Unite.AI Prompt Hub Crucially, **62% of multi-turn failures** occurred when models locked into suboptimal solutions early and stubbornly defended them, and **78% of initial errors propagated** through subsequent turns. Techfunnel Reasoning models (o3, DeepSeek-R1) showed no improvement — longer thinking only produced longer wrong answers. Prompt Hub The mitigation is to consolidate scattered context into single prompts where possible rather than drip-feeding across turns. Prompt Hub

The "lost in the middle" effect remains real but is more nuanced than originally described. The original Stanford/UC Berkeley paper documented a U-shaped performance curve where information in the middle of long prompts gets missed. arXiv +2 However, Chroma's more recent Context Rot study (July 2025) testing 18 models found that **all models degrade with longer inputs** even on trivial tasks like string repetition, Adaline ZenML and did not consistently observe the U-shape. Hamel's Blog They discovered that coherent, well-structured documents actually make retrieval harder than random text chunks — models get trapped following narrative arcs instead of finding specific information. PromptLayer Degradation begins around **3,000 tokens** for reasoning tasks, well below technical context window limits. Grit Daily +3

Instruction priority conflicts are fundamentally unreliable. The "Control Illusion" paper (2025) found that the system/user prompt separation fails to establish reliable instruction hierarchy — models exhibit strong inherent biases toward certain constraint types regardless of priority designation. arXiv arXiv Societal hierarchy framings (authority, expertise) show stronger influence than system/user roles. arXiv OpenAI's Instruction Hierarchy paper achieved 63% better resistance through explicit hierarchical training, but this requires model-level intervention the user cannot control. arXiv Iclr

Memory staleness creates confidently wrong outputs. LLM agents display an "experience-following property" where high similarity between current input and retrieved memory produces highly similar outputs — even when the memory is wrong or outdated. Combining selective addition and deletion strategies yielded a **10% absolute performance gain**. arXiv Emergent Mind Stale data acts as "poison" because models have no built-in mechanism to detect outdated retrieved snippets. Medium

Phase transition failures are common in multi-agent systems. The MAST study (UC Berkeley, 2025) found arXiv **41–86.7% failure rates** across seven production multi-agent frameworks. arXiv arXiv Galileo documented "planner" agents suddenly writing code instead of outlining it, agents withholding relevant context while pursuing divergent plans, Galileo AI and 62% of failures stemming from premature assumptions early in workflows.

Mitigation strategies for the 5-layer architecture

For coherence loss: consolidate all five layers into the prompt at each call rather than relying on the agent to recall previous turns' instructions. The 39% degradation from multi-turn instruction sharding makes this critical. For lost-in-the-middle: place the most important constraints at the beginning (system prompt) and end (workspace.md injected as the last synthetic tool result) of the assembled context. Medium For priority conflicts: repeat the most critical constraints in both the system prompt and workspace.md — redundancy across layers is a feature, not a bug, given unreliable priority hierarchies. Medium For memory staleness: implement timestamp tracking on workspace.md entries and periodic re-validation of stored decisions. For phase transitions: use explicit phase markers in the phase prompt, include a "you are now in [STRATEGIC/TACTICAL] phase" declaration, and validate phase-appropriate behavior through output

checking. Add validation checkpoints at phase boundaries — researchers found this reduced error propagation by **41% in coding tasks**. Techfunnel

---

Synthesis: consensus patterns and divergent opinions

Strongest consensus patterns across all sources

File system as external memory is the dominant pattern. Claude Code (CLAUDE.md), Manus (todo.md), Product Talk Devin (SUMMARY.md, CHANGELOG.md), and Factory.ai all converge on using structured markdown files as persistent external memory. Medium The agent reads and writes these files, treating the file system as unbounded storage while the context window serves as working memory. Letta This is the single most validated architectural pattern in the research.

Progressive disclosure with metadata routing is becoming an industry standard. Claude Code, GitHub Copilot, VS Code, Cursor (fetch_rules), and OpenHands (keyword-triggered microagents) GitHub all implement variants of the same pattern: lightweight metadata always loaded, full content loaded on demand. The three-tier approach (manifest → instructions → detailed resources) minimizes token costs while keeping information accessible. Honra

Structured compaction with anchored sections outperforms freeform summarization. Factory.ai's evaluation demonstrates this quantitatively. Systems that define explicit sections for compaction (intent, decisions, files modified, next steps) reliably preserve critical information, while freeform summarization drops details the summarizer considers "low-entropy" — including file paths, constraint lists, and specific numbers. Factory

Prompt caching requires strict static-to-dynamic ordering with append-only context. Every source — Anthropic docs, OpenAI docs, Manus production data, the "Don't Break the Cache" paper — converges on the same recommendation. Manus The user's 5-layer architecture maps naturally to this ordering: tools → system prompt → reference material → phase prompt + workspace → task + feedback.

Defense in depth for critical constraints is non-negotiable. No single mechanism reliably preserves constraints across all scenarios. The combination of system prompt placement + structured memory section + custom compaction prompts + re-injection from config + output validation provides the required reliability.

Areas where expert opinion diverges

Scaffold complexity versus model capability. SWE-agent's philosophy ("focus on the LM, not the scaffold") directly contradicts the elaborate architectures of Claude Code and Manus. GitHub The evidence suggests both are valid for different contexts: capable models (Claude 4, GPT-4.1) need less scaffolding, while multi-step workflows with persistence requirements benefit from structured architectures regardless of model capability.

Agent-initiated versus system-initiated retrieval. Anthropic advocates for agent autonomy in retrieval ("let the agent explore"), while the 56% non-invocation rate from Vercel's evaluations and Kamiwaza's finding of "premature action without grounding" suggest this is unreliable. The pragmatic consensus is hybrid: system-triggered retrieval for critical reference material, agent-triggered for exploratory context.

Context window size sweet spot. Aider recommends staying under 25K tokens. (Aider) Chroma's research shows degradation at 3K tokens for reasoning. (PromptLayer +2) Manus operates with 100:1 input-to-output ratios suggesting very large contexts. (Manus) (manus) The divergence likely reflects task type: reasoning tasks degrade earlier than retrieval or code-generation tasks.

Final architectural recommendations

The user's 5-layer architecture is well-conceived and aligns with production patterns. The key refinements based on this research:

1. **Order layers for caching**: tools → system prompt (Layer 1) → reference material (Layer 4, when loaded) → phase prompt (Layer 2) + workspace.md (Layer 3) → task + feedback (Layer 5). Place Anthropic cache breakpoints between each transition.
2. **Make reference material system-triggered, not purely agent-triggered**: When the orchestrator transitions to a new phase, it should automatically load the relevant reference material for that phase rather than trusting the agent to fetch it. This counters the 56% non-invocation failure rate.
3. **Structure workspace.md with anchored sections**: Use Factory.ai's pattern with explicit, named sections. Auto-seed a Pinned Instructions section from YAML config. Customize the compaction prompt to preserve pinned sections verbatim.
4. **Implement Manus's todo.md pattern**: Continuously rewrite a structured progress section at the end of workspace.md to exploit recency bias and keep the current objective in the model's active attention. (Medium) (manus)
5. **Add post-generation constraint validation**: For critical constraints like "every claim cites evidence," implement programmatic output checking independent of the LLM — this is the only approach that survives all failure modes.
6. **Keep the system prompt under 4K tokens**: This ensures it clears the minimum cacheable threshold for all Anthropic models (claude) while staying well under the point where instruction-following degrades. Put identity, memory model, guardrails, and the meta-instruction about pinned constraints here — nothing else.
7. **Monitor cache hit rates as a primary operational metric**: Following Manus's guidance, track (manus) (cache_read_input_tokens) / total input tokens as the single most important cost and performance indicator for the production system.